MEDAC
Instituto Oficial de Formación Profesional

Práctica 1 - Mi primera app Android: de la idea al proyecto con recursos

Marta González González

2ºDAM

Índice

Práctica 1 - Mi primera app Android: de la idea al proyecto con recursos **1**
1. Introducción **3**
2. Parte A: Análisis del desarrollo móvil **3**
2.1. Ventajas e inconvenientes del desarrollo móvil 3
2.2. Elección tecnológica: Nativo, Híbrido y Multiplataforma 5
2.3. Ciclo de vida de un Activity 7
3. Parte B: Especificación de la app **8**
3.1. Idea, público objetivo y objetivos medibles 8
3.2. Requisitos funcionales 10
3.3. Requisitos no funcionales 10
3.4. Casos de uso 11
4. Parte C: Entorno de desarrollo **14**
4.1. Creación del proyecto y estructura 14
4.2. Configuración de dispositivos virtuales (AVD) 17
4.3. Ejecución de la aplicación 18
5. Parte D: Recursos de la aplicación **19**
5.1. Textos e internacionalización 20
5.2. Imágenes e icono de la aplicación 22
5.3. Colores, estilos y tema 23
6. Conclusión personal **25**
6.1. Aprendizaje adquirido 25
6.2. Dificultades encontradas 26
7. Bibliografía **26**

MEDAC

Índice de tablas

1. Tabla comparativa entre desarrollo Nativo, Híbrido y Multiplataforma 6
2. Detalles técnicos de los dispositivos virtuales (AVD) configurados 17

Índice de ilustraciones

1. Diagrama del ciclo de vida de una Activity 7
2. Diagrama de casos de uso de la aplicación Querida Aiko 12
3. Conexión en código entre MainActivity.java y activity_main.xml 17
4. Ejecución de prueba de la aplicación en el emulador móvil Pixel 7 18
5. Ejecución de prueba de la aplicación en el emulador Pixel Tablet 19
6. Diseño inicial de las pantallas e icono de la aplicación en Figma 20
7. Comparativa de la aplicación traducida en español e inglés 21
8. Icono adaptativo colocado en el menú lanzador del emulador 22
9. Aspecto final de la aplicación ejecutándose en el emulador de móvil 24
10. Aspecto final de la aplicación ejecutándose en el emulador de tablet 25

1. Introducción

En este documento explico todo el desarrollo de mi primera práctica de PMDM, desde la idea inicial hasta dejar el proyecto montado en Android Studio. Más allá de programar la funcionalidad, el objetivo ha sido entender cómo funciona el desarrollo nativo en Java y XML, controlar la estructura del proyecto y aprender a gestionar todos los recursos

Para la app he creado “Querida Aiko”, un espacio de cartas y reflexiones nocturnas que funciona como una cápsula del tiempo. El funcionamiento se centra en redactar notas o pensamientos (normalmente por la noche) y asignarles una fecha y una hora concreta de apertura. Al guardarlas, el contenido se queda bloqueado automáticamente, impidiendo su lectura hasta que el sistema verifique que ha llegado el momento exacto programado. Una vez superado ese plazo, la nota se desbloquea para poder leerla al completo

2. Parte A: Análisis del desarrollo móvil

2.1. Ventajas e inconvenientes del desarrollo móvil

Desarrollar una aplicación para móviles tiene cosas buenas pero también cosas que no te encuentras cuando programas para un ordenador normal. La mayor ventaja que le veo es la disponibilidad casi total porque solemos llevar el teléfono encima todo el día. Pensando en mi proyecto, que funciona como una cápsula del tiempo, esto es importante. Te da la libertad de escribir un pensamiento o guardar una promesa en el instante exacto en el que lo estás sintiendo, sin tener la obligación de llegar a casa para encender el PC. Por otro lado está la ventaja de acceder a herramientas del propio sistema, pudiendo programar notificaciones que avisen al usuario justo el día que toca abrir esa carta antigua

Pero también existen inconvenientes importantes debido a las limitaciones físicas de los teléfonos. En el desarrollo de esta aplicación tendré que tener muy en cuenta sobretodo esto:

El consumo de batería: Es uno de los recursos más valiosos y importantes en cualquier móvil. Como mi aplicación necesita avisar en fechas concretas del futuro para leer las cartas guardadas, no puedo dejar procesos ejecutándose en segundo plano continuamente para comprobar qué día es. Si hiciera eso la batería duraría muy poco. La solución que aplicaré será aprovechar el gestor de alarmas de Android para que la app solo reaccione en el momento exacto en el que tiene que enviar el aviso

El tamaño de la pantalla: En el ordenador suele sobrar espacio visual, mientras que en un móvil hay que ir a lo importante para no saturar al usuario ni que se le canse la vista. El objetivo de la app es que la gente escriba textos largos y personales. Por eso necesito crear un diseño muy limpio. Y los botones deben tener un tamaño cómodo para pulsarlos con el pulgar y la tipografía tiene que leerse perfectamente sin esfuerzo

La enorme variedad de dispositivos: Hoy en día cada persona tiene un móvil de una marca distinta con resoluciones y proporciones de pantalla totalmente diferentes. Si no tengo cuidado al diseñar y lo hago pensando solo en mi teléfono, la lista de promesas bloqueadas podría verse deformada o incluso cortada en pantallas más pequeñas. Para evitar esto utilizaré un diseño responsive a la hora de organizar las vistas, y así aseguro que todos los elementos se adapten automáticamente al tamaño del dispositivo

El uso del teclado virtual: Al escribir en un ordenador se tiene un teclado físico y se puede ver la pantalla completa. En el móvil, cuando escribimos una carta larga, el teclado táctil aparece de golpe y tapa casi la mitad de la pantalla. Para mi app esto es un reto, así que tendré que configurar el diseño para que la pantalla se desplace hacia

arriba automáticamente cuando se abra el teclado. Así me aseguro de que el usuario siempre pueda ver bien lo que está escribiendo sin sentirse agobiado

La memoria y el almacenamiento: Los móviles no tienen tanta potencia de procesamiento ni espacio libre como un ordenador. Si empiezo a cargar imágenes de calidad muy alta y también muy pesadas para decorar la interfaz, es muy probable que la aplicación acabe yendo a tirones o que el sistema la cierre de golpe para liberar la memoria RAM. Para que todo funcione de manera fluida y el proyecto pese lo mínimo posible, usaré imágenes vectoriales en los iconos y adornos. Este formato no pierde calidad y ocupa muy poco

La conectividad: Los móviles dependen de la cobertura y a veces se pierde la conexión a internet si estás en el metro o de viaje. Como mi idea es que el usuario pueda escribir lo que siente en cualquier momento, desarrollaré la aplicación para que funcione perfectamente de forma local sin necesidad de tener internet

Seguridad y permisos: En un ordenador normalmente tienes acceso a todo, pero en el móvil hay que pedir permiso al usuario para usar ciertas cosas por seguridad. En mi app, tendré que pedirle permiso expresamente a la persona para poder enviarle notificaciones, ya que si las deniega, la cápsula del tiempo no podrá avisarle de que tiene una carta lista para leer

2.2. Elección tecnológica: Nativo, Híbrido y Multiplataforma

A la hora de plantear cómo desarrollar una aplicación, existen diferentes caminos. Cada uno tiene sus ventajas y sus inconvenientes dependiendo del presupuesto y de lo que necesitemos hacer. En esta tabla comparo las tres opciones principales:

TIPO DE DESARROLLO	NATIVO	HÍBRIDO	MULTIPLATAFORMA
Lenguajes utilizados	Java o Kotlin para Android, y Swift para iOS	HTML, CSS y JavaScript	Dart, C# o JavaScript
Rendimiento	Muy alto. La app va fluida porque habla el idioma directo del móvil	Más bajo. A veces va a tirones porque en realidad es una web metida en una app	Alto. Se acerca mucho al nivel del desarrollo nativo
Acceso al hardware (sensores, cámara...)	Total. Se tiene acceso completo a todas las herramientas del sistema sin restricciones	Limitado. Depende de plugins externos para usar cosas como la cámara o el GPS	Bueno. Utiliza librerías que hacen de puente para acceder a los componentes del teléfono
Coste y tiempo	Alto. Hay que programar una app desde cero para Android y otra distinta para iOS	Bajo. Se escribe el código una vez y sirve para web y para los distintos móviles	Medio. Es un solo código, pero suele hacer falta ajustar detalles para cada sistema operativo
Experiencia de usuario (UX)	Excelente. Se usan los botones y menús propios del sistema, por lo que resulta muy familiar	Regular. A veces la interfaz no se siente como una app real y puede resultar extraña	Muy buena. Permite hacer diseños muy personalizados que se ven casi igual en todos los móviles
Ejemplo de framework	Android SDK, iOS SDK	Apache Cordova, Ionic	Flutter, React Native

Justificación de mi elección:

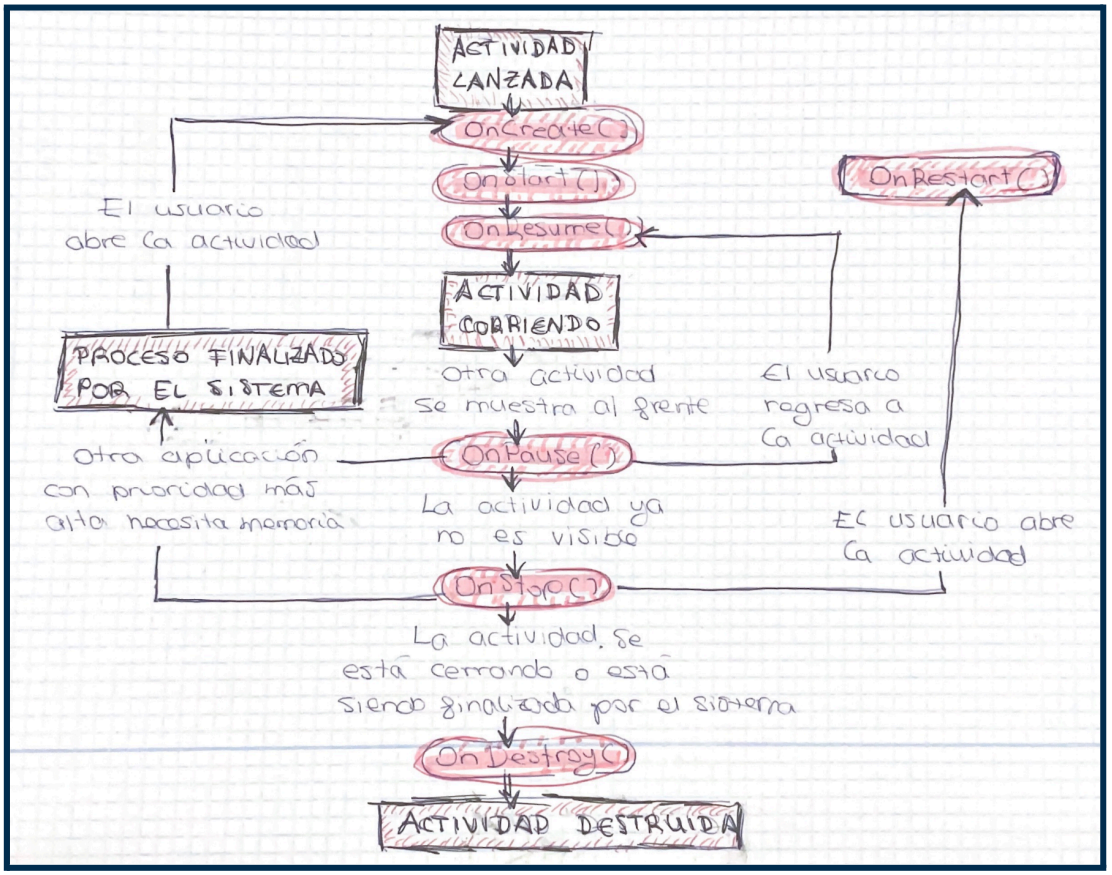
Para hacer la app he elegido el desarrollo nativo en Android usando Java para la lógica y XML para el diseño. Aunque usar algo multiplataforma podría estar bien si en un futuro quisiera sacar la app también para otros sistemas operativos, para este proyecto en concreto el nativo me parece la mejor opción y la más lógica

Lo principal es que la aplicación necesita usar las alarmas del propio móvil para avisar de que se ha desbloqueado una carta. Si esto se hiciera con un desarrollo híbrido, habría que buscar y depender de plugins. El problema de eso es que muchas veces dan fallos o los dejan de actualizar, y prefiero no arriesgarme con la función principal de la app. Trabajando directamente en nativo me aseguro de que el aviso va a saltar perfecto sin depender de nada externo

Aparte de esto, al usar Android Studio me resulta mucho más cómodo tener todo organizado. Es muy fácil gestionar las carpetas para guardar las imágenes vectoriales, la paleta de colores y los textos traducidos. Así también aplico directamente lo que hemos estado viendo en clase con la plantilla

2.3. Ciclo de vida de un Activity

El ciclo de vida de una Activity son los diferentes estados por los que pasa una pantalla de la aplicación desde que se abre hasta que se cierra. Como se puede ver en el diagrama que he dibujado, el proceso va pasando por distintos métodos dependiendo de lo que haga el usuario o el propio sistema



Para mi app, he analizado qué pasaría en el ciclo de vida en estas tres situaciones concretas, imaginando que el usuario está a la mitad de escribir una carta:

MEDAC

El usuario recibe una llamada: La pantalla de la app pasa a un segundo plano porque entra la pantalla de la llamada. Mirando el diagrama, aquí pasaríamos de la "actividad corriendo" a ejecutar los métodos `onPause()` y luego `onStop()` al dejar de ser visible. Como el usuario puede llevar un rato escribiendo algo importante, en mi app aprovecharía este momento para guardar un borrador automático. Así aseguro que no pierde lo escrito si la llamada dura mucho rato y el sistema decide matar el proceso por falta de memoria

El usuario gira el móvil: Al girar la pantalla, el sistema destruye la vista por completo y la vuelve a crear para adaptarla al nuevo tamaño. Pasa por `onPause()`, `onStop()`, `onDestroy()` y vuelve a arrancar de cero con `onCreate()`, `onStart()` y `onResume()`. En mi app el diseño se adaptaría solo gracias a usar un diseño responsive a la hora de organizar las vistas. Pero además tendría que usar el método `onSaveInstanceState()` para que el texto que estaba escribiendo vuelva a aparecer en su sitio y no se le borre de golpe al girar el teléfono

El usuario pulsa el botón atrás: Esto le dice al sistema que el usuario quiere salir de la pantalla actual. Siguiendo el diagrama, se ejecutarían los métodos `onPause()`, `onStop()` y por último `onDestroy()` para destruirla del todo. Si el usuario hace esto mientras está escribiendo una promesa, mi aplicación no le dejaría salir directamente. Le saltaría un aviso preguntando si está seguro de salir sin guardar, para evitar que pierda la carta por darle sin querer al botón

3. Parte B: Especificación de la app

3.1. Idea, público objetivo y objetivos medibles

Mi aplicación se llama "Querida Aiko" y funciona como una cápsula del tiempo emocional inspirada en el manga “Oyasumi Punpun”. La idea principal es que la persona pueda escribir promesas, desahogos, pensamientos, etc. Y dejarlos bloqueados con una alarma, de forma que no se puedan volver a leer hasta una fecha futura

elegida por el mismo usuario. El público objetivo son jóvenes de entre 15 y 25 años que buscan un espacio así más íntimo y seguro para usarlo a modo de diario personal, y poder recordar en un futuro un pensamiento o sentimiento relacionado que tenían en ese entonces. Además, está pensada directamente para los fans de Inio Asano, ya que hace una pequeña referencia al manga

Para comprobar que la aplicación cumple su función, me he marcado estos cuatro objetivos medibles:

1. Conseguir que el usuario tarde menos de 1 minuto en escribir y dejar bloqueada una promesa nueva, asegurando así que el proceso es directo y no da pereza usarlo

2. Lograr que el 100% de las cartas guardadas lancen su aviso el día exacto programado, comprobando que el sistema de alarmas de Android funciona sin fallos

3. Asegurar que la pantalla principal cargue y muestre la lista de todas las promesas en menos de 2 segundos, incluso si hay más de 50 elementos guardados en la aplicación

4. Alcanzar que la persona que se descargue la app escriba un mínimo de 3 cartas durante su primera semana usándola, demostrando que la idea engancha y resulta útil

3.2. Requisitos funcionales

- RF-01: El usuario debe poder redactar una carta o pensamiento nuevo y guardarlo en la aplicación. **Prioridad:** alta

- RF-02: La aplicación tiene que permitir elegir una fecha exacta en el futuro para programar cuándo se podrá abrir la carta. **Prioridad:** alta

- RF-03: El sistema debe bloquear totalmente la lectura de cualquier carta si la fecha que se eligió todavía no ha llegado. **Prioridad**: alta

- RF-04: El usuario podrá ver una lista principal con todas sus cartas guardadas, diferenciando visualmente cuáles están bloqueadas y cuáles están listas para leer. **Prioridad:** alta

- RF-05: La aplicación lanzará una notificación en el móvil del usuario justo el día en el que una promesa antigua pase a estar desbloqueada. **Prioridad:** media

- RF-06: El usuario tendrá la opción de borrar de forma definitiva cualquier carta de su lista por si se arrepiente o ya no quiere guardarla. **Prioridad:** media

- RF-07: El usuario podrá cambiar manualmente el idioma de la aplicación entre español e inglés mediante un botón en los ajustes. **Prioridad**: baja

3.3. Requisitos no funcionales

- RNF-01 (Privacidad verificable): La aplicación funcionará 100% de manera local y no necesitará acceso a internet

- RNF-02 (Fiabilidad): El sistema realizará un autoguardado en segundo plano exactamente cada 30 segundos mientras el usuario escribe. Esto se podrá verificar escribiendo un texto, cerrando la app de golpe a los 35 segundos desde los procesos del móvil, y comprobando al volver a entrar que el texto sigue ahí guardado

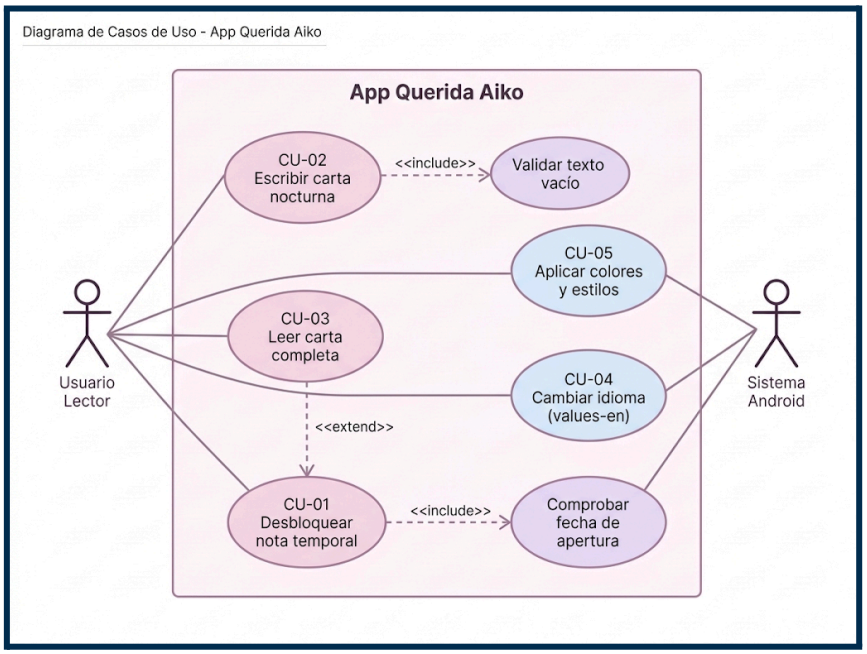
- RNF-03 (Usabilidad visual): El diseño de la interfaz no se deformará ni cortará texto si el usuario pone el tamaño de fuente al máximo en los ajustes de su móvil. Esto será comprobable usando solo unidades sp (y no fijos) para las tipografías

- RNF-04 (Rendimiento): La pantalla principal mostrará la lista de promesas en menos de 2 segundos de reloj, algo que se podrá medir con un cronómetro

- RNF-05 (Compatibilidad): La aplicación se podrá instalar y abrir sin que lance ningún error de compatibilidad en un emulador configurado exactamente con la versión Android 7.0 (API 24). He elegido esta versión mínima porque hoy en día está presente en casi el 100% de los teléfonos Android activos, por lo que me aseguro de que prácticamente cualquier persona pueda instalar la app aunque tenga un móvil antiguo, pero contando ya con soporte completo para el sistema de alarmas, notificaciones y diseño de vistas que necesita mi proyecto

3.4. Casos de uso

En esta sección explico cómo se relaciona la persona que usa mi aplicación con la app. La idea principal es ver qué puede hacer el usuario y cómo el propio sistema operativo del móvil le ayuda por detrás para que todo funcione de la forma esperada:



Análisis y diseño del diagrama:

Para construir el diagrama de casos de uso de manera limpia y organizada, he definido dos actores principales situados a cada lado. A la izquierda se encuentra el usuario lector, que es quien abre la aplicación para escribir sus cartas, leerlas cuando toque o cambiar los ajustes visuales y de idioma. A la derecha he colocado el sistema Android, ya que la aplicación necesita consultar datos internos del teléfono, como la hora exacta del dispositivo para calcular los bloqueos temporales o el idioma configurado en el sistema

Todo lo que forma parte de mi proyecto queda dentro de una caja general titulada con el nombre de la app. Dentro de este espacio he agrupado las funciones clave mediante varios casos de uso principales. El núcleo de la aplicación es el desbloqueo de notas temporales, que se complementa con la redacción de la carta nocturna y la lectura del contenido completo una vez que el tiempo se ha cumplido. Además, he añadido opciones de configuración para cambiar el idioma al inglés y aplicar una paleta de colores personalizada

Para que el comportamiento de la aplicación sea realista, el diagrama muestra por ejemplo, que al guardar una carta el sistema compruebe obligatoriamente que el campo de texto no esté vacío, y al intentar abrir una nota comprueba si la fecha actual ya ha alcanzado el límite establecido

Fichas detalladas de casos de uso:

Para detallar a fondo cómo se comportan las funciones más importantes de la aplicación, desarrollo a continuación las fichas técnicas correspondientes a los dos casos principales.

FICHA DE CASO DE USO: **CU-01 — Desbloquear nota temporal**

Este caso de uso se activa cuando el usuario se encuentra en la pantalla principal y pulsa sobre una de las cartas que sigue bloqueada, pero cuya fecha y hora de apertura en teoría ya deberían haber llegado. El flujo principal comienza cuando la persona hace clic encima de la nota elegida. En ese instante, la aplicación se comunica de manera interna con el dispositivo para consultar el día y la hora exactos que marca el reloj en ese momento. A continuación, el sistema contrasta la hora actual con el límite temporal que quedó guardado junto a la carta cuando fue escrita. Si el tiempo transcurrido es igual o superior a la fecha programada, la aplicación cambia el estado visual de la nota, la desbloquea y le permite al usuario entrar para leer su contenido completo de forma permanente

Como flujo alternativo, si el móvil indica que todavía es pronto y no se ha alcanzado la fecha configurada, la aplicación muestra un pequeño aviso en pantalla informando de que la carta sigue cerrada y que toca esperar un poco más, devolviendo al usuario a la pantalla principal sin alterar el bloqueo

RESUMEN:

- **Actor**: Usuario lector y Sistema Android (reloj interno del dispositivo)

- **Precondición**: Estar en la pantalla principal con al menos una nota guardada en estado bloqueado

- **Flujo principal:** El usuario pulsa la nota, el sistema comprueba que la fecha y hora actual igualan o superan la programada y muestra el contenido de la carta

- **Flujo alternativo:** Si aún no ha llegado la fecha indicada, la app impide abrirla, muestra un aviso de que sigue cerrada y vuelve a la pantalla principal

- **Postcondición:** La nota queda desbloqueada de forma permanente para su lectura (o sigue bloqueada si salta el flujo alternativo)

FICHA DE CASO DE USO: **CU-02 — Escribir carta nocturna**

Este segundo caso de uso cubre todo el proceso de creación y almacenamiento de una nueva reflexión dentro de la aplicación. El proceso arranca cuando el usuario entra en la sección de redacción, escribe su pensamiento en el cuadro de texto habilitado para ello y selecciona la fecha y la hora exacta en la que desea que esa carta pueda volver a abrirse en el futuro. Al pulsar el botón de guardar, la aplicación ejecuta una comprobación automática para verificar que el texto introducido contiene información válida y no se ha dejado completamente en blanco. Una vez superada esta validación, los datos se almacenan de manera local en el teléfono junto con los metadatos temporales, y la aplicación muestra un aviso rápido de confirmación antes de devolver al usuario a la pantalla de inicio.

Si ocurriera el caso alternativo de que el usuario intente guardar la carta sin haber escrito nada o dejando el campo con espacios vacíos, el proceso se frena al momento. La aplicación le muestra un mensaje advirtiéndole de que el contenido no puede estar en blanco y lo deja de nuevo en la pantalla de edición para que pueda redactar un texto válido antes de reintentar el guardado

RESUMEN:

- **Actor**: Usuario lector y Sistema Android (almacenamiento local y selector de fecha)

- **Precondición**: Haber pulsado el botón de añadir y encontrarse en la pantalla de nueva promesa

- **Flujo principal:** El usuario redacta la nota, elige la fecha y hora de apertura, pulsa guardar y el sistema valida el texto y lo almacena bloqueado

- **Flujo alternativo:** Si el texto está vacío o solo tiene espacios en blanco, la app detiene el guardado, lanza un aviso y mantiene al usuario en la pantalla de edición

- **Postcondición:** La carta se guarda bloqueada en la memoria local con su fecha de desbloqueo y se añade a la lista de la pantalla de inicio

4. Parte C: Entorno de desarrollo

4.1. Creación del proyecto y estructura

Para iniciar el desarrollo de la aplicación, he creado un nuevo proyecto en Android Studio utilizando la plantilla "Empty Views Activity". Siguiendo las instrucciones de la actividad, he configurado el proyecto con lo siguiente:

- **Nombre del proyecto:** PMDM_P1_MartaGonzalez
- **Nombre del paquete:** es.medac.martagonzalez.app
- **Lenguaje:** Java
- **Minimum SDK:** API 24 (Android 7.0), para garantizar una alta compatibilidad con dispositivos antiguos

Una vez que terminó de cargar Gradle, Android Studio me generó la estructura de carpetas típica de cualquier proyecto. Básicamente, he organizado el trabajo separando la lógica del diseño. Los archivos principales son los siguientes:

`manifests/AndroidManifest.xml`: Es como el archivo de configuración central de la app. Aquí es donde le digo a Android cómo se llama la aplicación, qué icono utiliza y, muy importante, declaro las pantallas (activities) que he creado y cuál es la primera que se tiene que abrir

`java/es.medac.martagonzalez.app/MainActivity.java`: Aquí es donde va el código fuente. En esta clase programo toda la lógica en Java, es decir, qué pasa cuando el usuario interactúa con la pantalla y cómo se comporta la aplicación por detrás

`res/` (recursos): Esta carpeta es para toda la parte visual y de configuración. Está pensada para no mezclar el código Java con el diseño. Tiene varias subcarpetas importantes:

- `layout/`: Aquí está el archivo `activity_main.xml`, que es donde creo la estructura de la interfaz (cajas de texto, botones, listas) usando etiquetas XML

- `drawable/` y `mipmap/`: En drawable se guardan las imágenes o gráficos base que se necesiten, y en mipmap se guardan las distintas resoluciones del icono de la app para que no se vea borroso en ninguna pantalla sin importar su tamaño

- `values/`: Es súper útil para mantener el código limpio. Aquí tengo `strings.xml` para guardar todos los textos (lo que me facilitará traducir la app al inglés), `colors.xml` para la paleta de colores hexadecimales, `dimens.xml` para centralizar todas las medidas de márgenes y textos, y los archivos `styles.xml` y `themes.xml` para los estilos y el tema visual.

Así evito escribir textos, colores o tamaños fijos directamente en el código de las pantallas

- `xml/`: Aquí Android Studio me ha generado automáticamente archivos como `backup_rules.xml` y `data_extraction_rules.xml`, que sirven para configurar cómo se gestionan las copias de seguridad de los datos de la app

`keepRules/`: Es una carpeta que se crea para configurar reglas por si más adelante necesito comprimir el código antes de publicar la aplicación

`Gradle Scripts/build.gradle`: Son los archivos que se encargan de compilar el proyecto. Aquí es donde defino las versiones de Android que estoy usando y donde añadiré dependencias o librerías externas si las necesito en el futuro

Relación entre la activity y su layout:

En Android, el código Java y el diseño XML trabajan siempre juntos. Pero, por defecto son independientes y no se conocen. Para conectarlos, utilizo la instrucción `setContentView(R.layout.activity_main)` dentro del método `onCreate()` de mi clase Java

Lo que hace exactamente esta línea es un proceso que se conoce como inflado. Que básicamente, coge el diseño estático que he dejado preparado en el archivo XML, lo convierte en elementos reales dentro de la memoria del móvil y los muestra por pantalla

Una vez que el diseño ya es visible, necesito poder interactuar con él desde el código. Por ejemplo, si quiero que un botón responda al pulsarlo o necesito leer lo que el usuario

ha escrito en un campo de texto, utilizo el método `findViewById()`. Esta función me permite buscar cualquier elemento visual por el ID que le puse en el XML y conectarlo con una variable en Java. De esta forma, logro vincular la parte gráfica con la lógica y ya tengo el control total para darle funcionalidad a la interfaz

activity_main.xml MainActivity.java

```java
package es.medac.martagonzalez.app;

import ...

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_main);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), ( View v, WindowInsetsCompat insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });
    }
}
```

4.2. Configuración de dispositivos virtuales (AVD)

Para poder probar cómo se va a ver mi aplicación en pantallas de distintos tamaños, he usado el “Device Manager” de Android Studio para montar dos emuladores, un móvil y una tablet

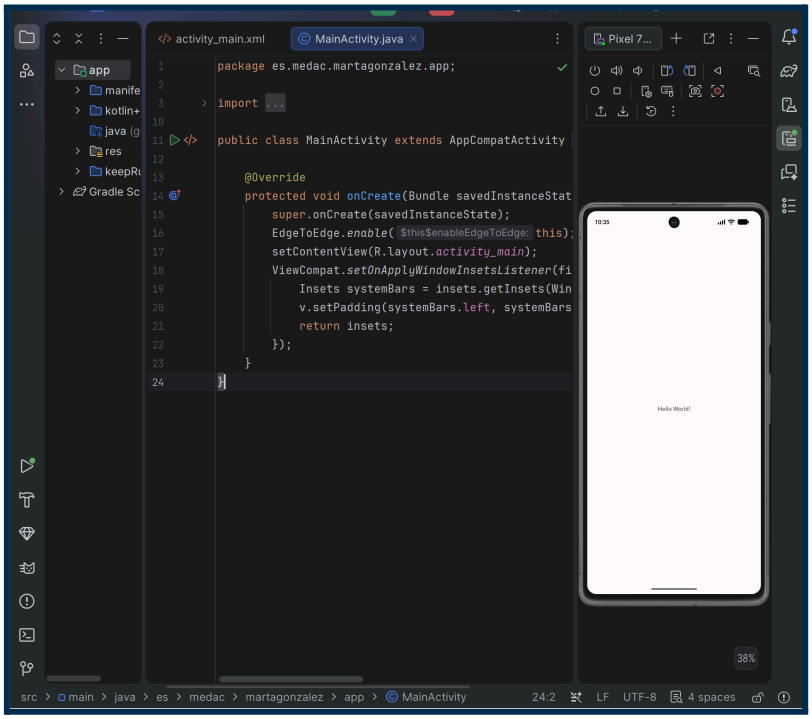
Aunque en los requisitos iniciales había puesto que la app usaría una versión antigua (API 24) para que fuera compatible con casi cualquier móvil, a la hora de crear los emuladores he preferido tirar por unas versiones más modernas que me recomendaba el propio programa (API 35 y 37.2). Así me aseguro de que los emuladores vayan mucho más fluidos en mi equipo y me evito errores raros de compatibilidad. Aquí dejo los detalles técnicos de los dos dispositivos que he configurado:

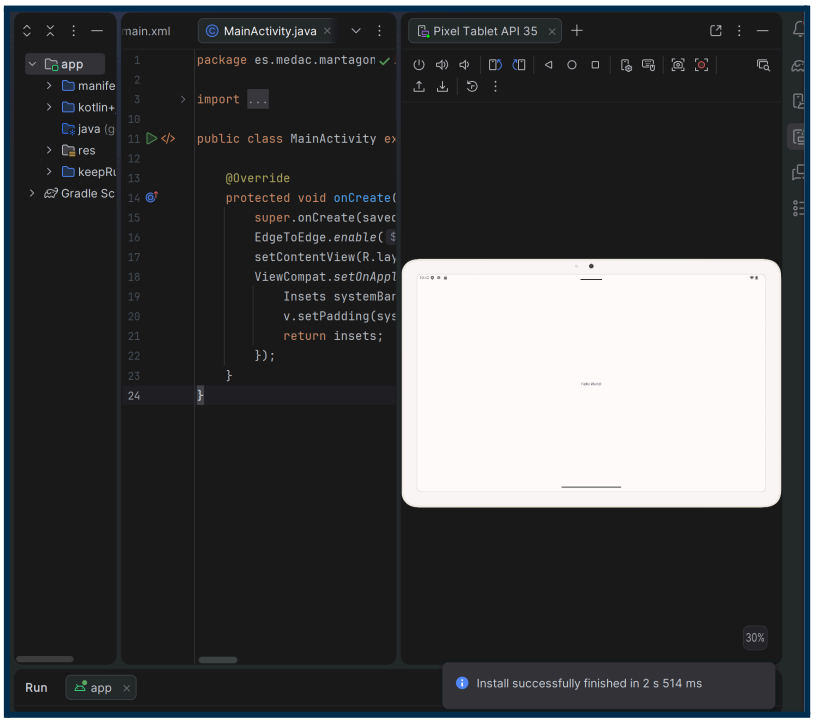
TIPO DE DISPOSITIVO	MODELO (AVD)	RESOLUCIÓN	DENSIDAD DE PÍXELES	VERSIÓN DE API
Móvil	Pixel 7	1080 x 2400 px	420 dpi (xxhdpi)	API 37.2
Tablet	Pixel Tablet	2560 x 1600 px	320 dpi (xhdpi)	API 35

4.3. Ejecución de la aplicación

Ya con los dos emuladores creados y listos, toca el último paso, que es arrancar la app para comprobar que toda la configuración está bien. He compilado el proyecto y lo he ejecutado primero en el Pixel 7 y luego en la tablet. Como la app está recién creada y todavía no he pasado a código el diseño que he hecho en Figma, lo único que sale en pantalla es el fondo blanco con el mensaje de "Hello World" que genera Android Studio por defecto. Lo importante de esta prueba es ver que la aplicación compila a la primera, se instala y se abre sola en los dos dispositivos sin que se cierre de golpe ni salte ningún error

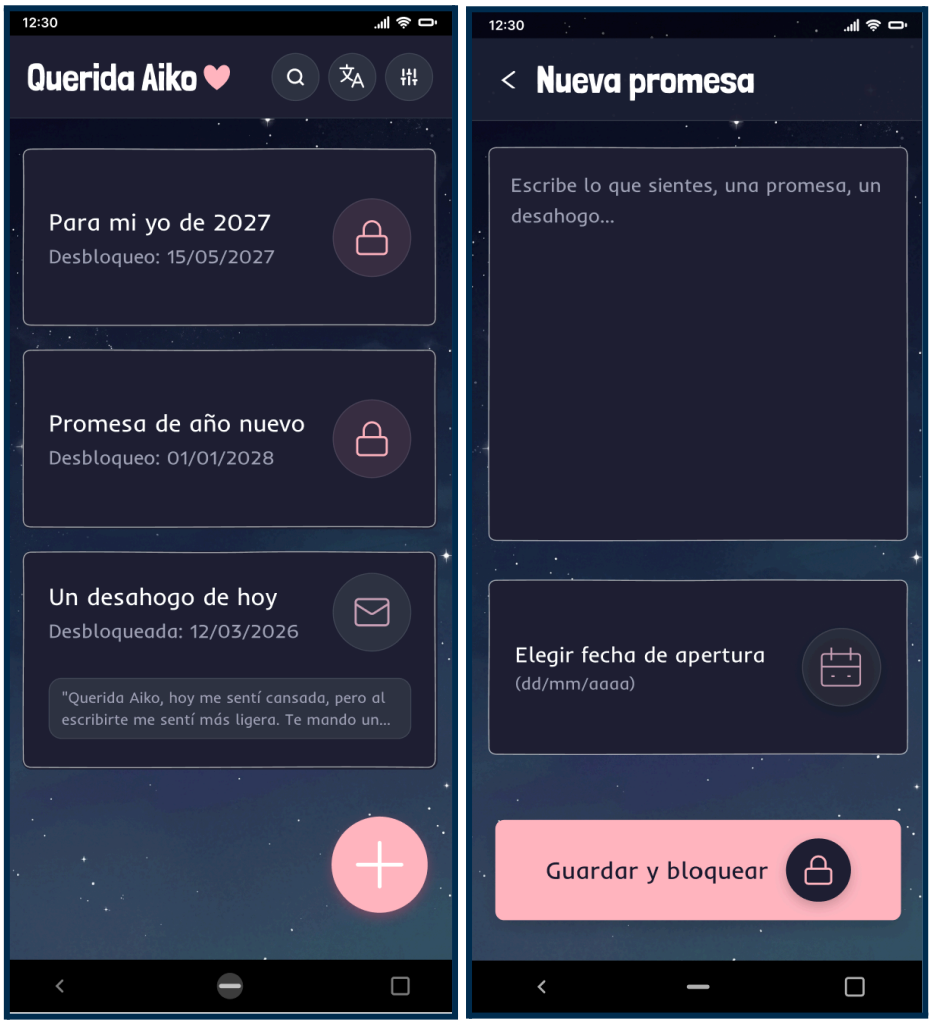
Aquí abajo dejo las capturas donde se ve la app funcionando en los dos emuladores:


activity_main.xml
MainActivity.java
Pixel 7...
app
package es.medac.martagonzalez.app;
import ...
public class MainActivity extends AppCompatActivity
@Override
protected void onCreate(Bundle savedInstanceStat
super.onCreate(savedInstanceState);
EdgeToEdge.enable($this$enableEdgeToEdge: this);
setContentView(R.layout.activity_main);
ViewCompat.setOnApplyWindowInsetsListener(fi
Insets systemBars = insets.getInsets(Win
v.setPadding(systemBars.left, systemBars
return insets;
});
}
}
Hello World!
38%
src > main > java > es > medac > martagonzalez > app > MainActivity
24:2
LF
UTF-8
4 spaces


main.xml
MainActivity.java
Pixel Tablet API 35
app
package es.medac.martagon
import ...
public class MainActivity ex
@Override
protected void onCreate(
super.onCreate(savec
EdgeToEdge.enable(
setContentView(R.lay
ViewCompat.setOnAppl
Insets systemBar
v.setPadding(sys
return insets;
});
}
}
30%
Run
app
Install successfully finished in 2 s 514 ms

5. Parte D: Recursos de la aplicación

Para diseñar esta app, primero estuve montando la idea en Figma para ver cómo quería que quedase toda la parte visual y luego ya me puse a pasarlo a código en Android Studio. Este es el diseño que realicé en Figma, y el icono de la aplicación:





5.1. Textos e internacionalización

1. Para empezar con los recursos de la app, lo primero que hice fue ordenar todos los textos para no dejar nada escrito a mano en las pantallas, porque eso en Android está mal visto y da fallos. Así que metí todo en el archivo `strings.xml`. En total puse 14 cadenas diferentes, con nombres sencillos

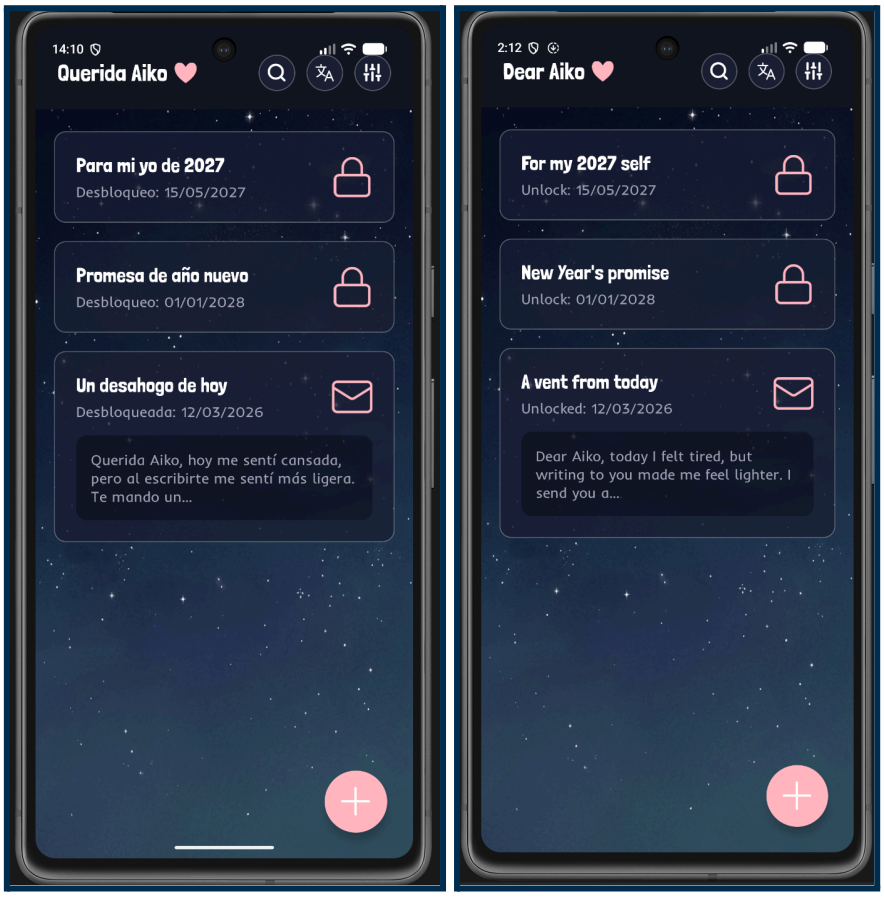
2. Para cumplir con lo de meter un parámetro, usé la estructura de porcentaje en el texto de la fecha: Desbloqueo: %1$s. Esto lo hice así para que luego, desde el código de la `MainActivity`, le pueda meter la fecha exacta sin tenerla fija en el diseño. Y para que mientras estoy editando el XML en el ordenador no se quede la pantalla fea o vacía, le puse el tools:text

3. Luego, para la parte de los idiomas, monté otra carpeta llamada values-en con su propio `strings.xml` traducido. Lo bueno de hacerlo así es que Android lo detecta solo, si le cambias el idioma al móvil o al emulador a inglés, los textos de la app se cambian al momento sin tener que programar nada más

4. Además de las cadenas normales y de los parámetros con formato (%1$s), también he añadido dentro de `strings.xml` un array de textos (string-array) llamado categorias_mensajes para clasificar los tipos de notas (Desahogo, Promesa, Carta al futuro y Pensamiento fugaz), un recurso de plurales (plurals)

para mostrar si falta un día o varios días para abrir la carta, y las descripciones de accesibilidad (contentDescription) para los iconos interactivos. Todo esto lo he traducido también al completo en el archivo `values-en/strings.xml`

Cómo se ve la app en los dos idiomas:



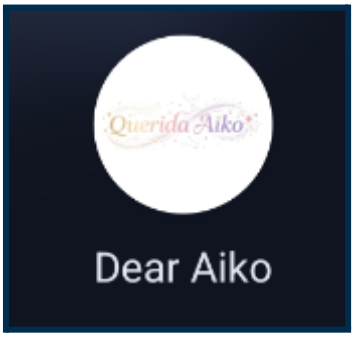
5.2. Imágenes e icono de la aplicación

Para decorar la app y que no se quedara con un fondo soso, metí un par de imágenes en la carpeta `res/drawable`. Por un lado, coloqué una imagen de mapa de bits para el fondo de pantalla de la app de las estrellas, y luego, añadí los iconos que saqué de Figma (como el candado, la carta, la lupa...) en formato vectorial

Esto lo hice así por lógica de rendimiento y peso, ya que el mapa de bits va bien para los fondos grandes donde hay muchos detalles y degradados de colores (como el cielo), porque al final es una foto que se queda fija. Pero, para los iconos pequeños y los botones, usar svgs es mil veces mejor porque pesan poquísimo, no pierden calidad por mucho que amplíes la pantalla y evitan que la aplicación ocupe más espacio del que se necesita en el almacenamiento del móvil

Además, para el icono que sale en el menú del móvil, creé uno propio usando la herramienta de image Asset de Android Studio. Y con esto generé el icono adaptativo, que se guardó automáticamente en todas las carpetas mipmap con sus diferentes tamaños, así aparece bien enlazado en el `AndroidManifest.xml` y se ve redondo o cuadrado según el terminal

Cómo se ve el icono ya colocado en el lanzador del emulador:



5.3. Colores, estilos y tema

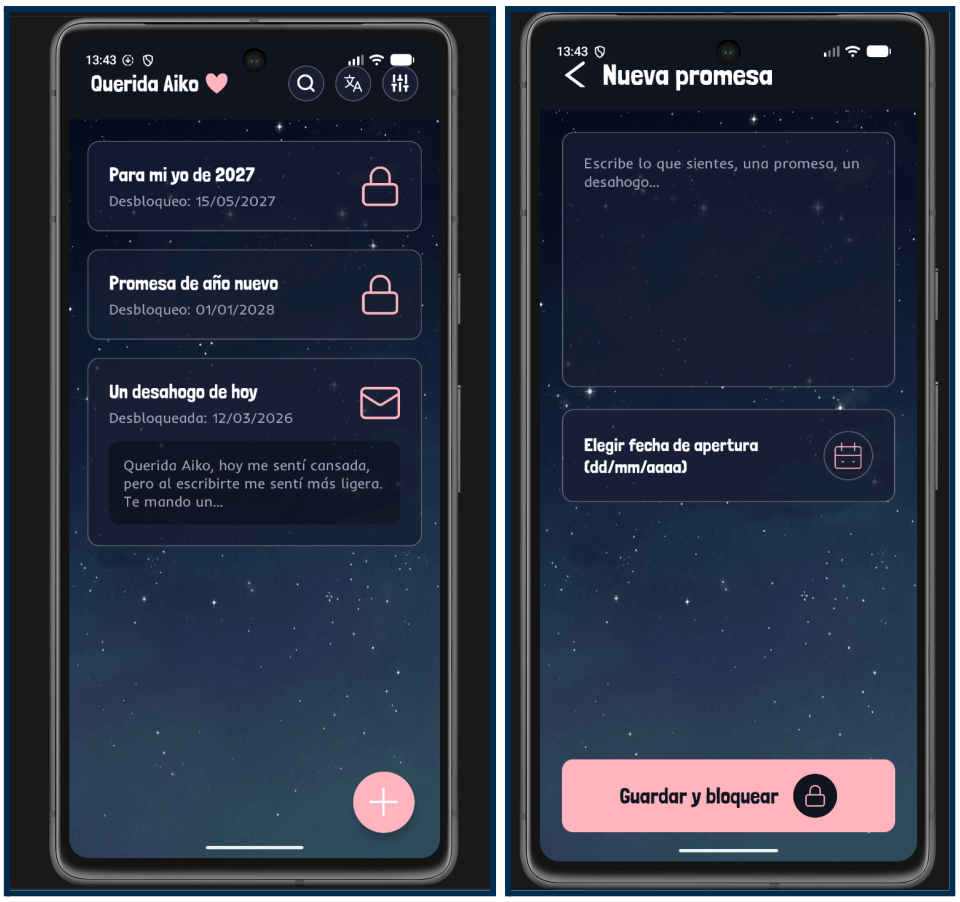
Para definir los colores de la app estuve montando la paleta en el archivo `colors.xml`. Puse 6 colores en total y a todos les di nombres funcionales para saber exactamente para qué los uso sin liarme. Metí “color_fondo_principal” para el fondo general, “color_fondo_tarjetas” y “color_fondo_desahogo” para los bloques donde van los textos, “color_acento_interfaz” para el rosa que destaca en botones y elementos clave, y “color_texto_primario” con “color_texto_secundario” para los títulos y descripciones

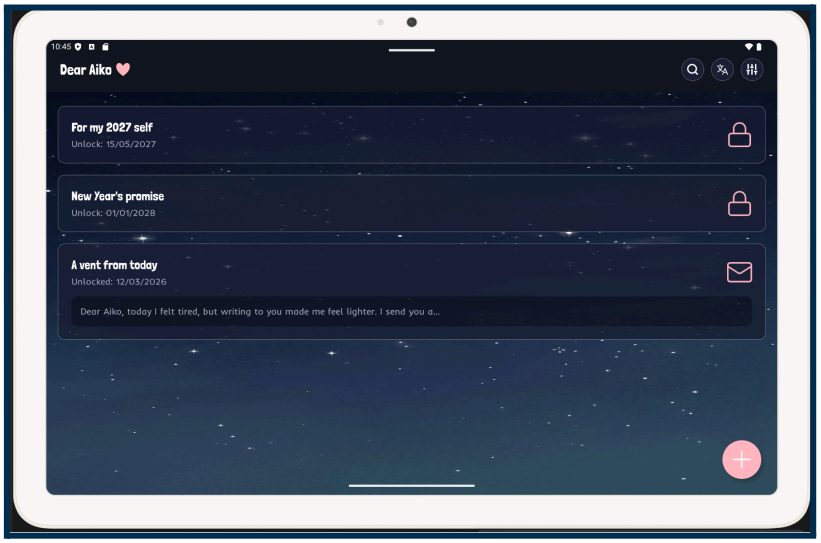
Elegí esta combinación de tonos oscuros y acentos en rosa pensando en la usabilidad real en las pantallas móviles. Un fondo oscuro ayuda a reducir la fatiga visual cuando usas la app con poca luz y además ahorra batería en pantallas OLED al apagar los píxeles negros. El tono rosa de acento lo utilicé solo en los elementos interactivos importantes como el botón flotante o las cerraduras para que el usuario identifique al instante dónde puede pulsar sin perder tiempo buscando

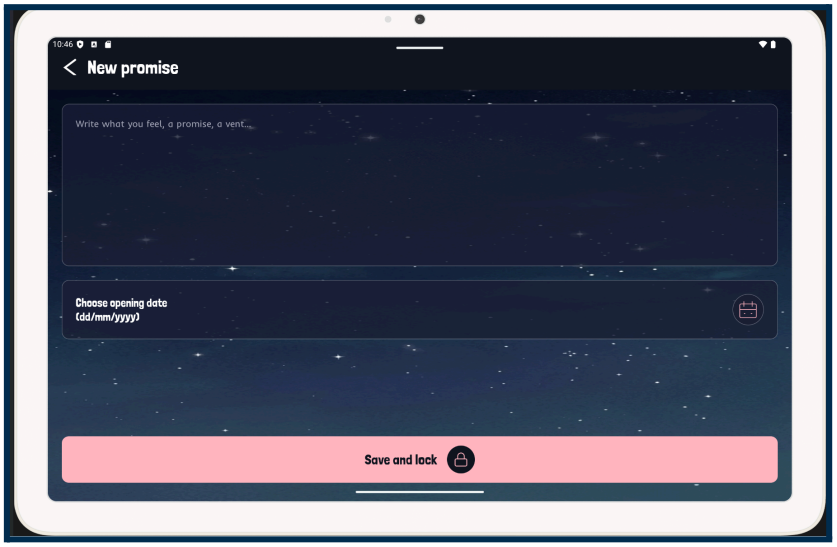
Con la tipografía y los estilos, descargué las fuentes “Londrina Solid” para los títulos y “Inder” para el texto, la primera aporta un toque estético personal y llamativo que encaja con la temática de las notas, mientras que la segunda garantiza una lectura limpia en los párrafos largos. Para no repetir código en las pantallas creé un par de estilos propios en `styles.xml` que llamé “EstiloTituloAiko” y “EstiloTextoSecundario” combinándolos con estas fuentes y reutilizándolos en los componentes de la interfaz. Para las medidas y los textos usé estrictamente dp en los tamaños y márgenes de las vistas y sp en las fuentes para que Android escale los textos bien si el usuario cambia la configuración de accesibilidad del móvil

Todas estas medidas en dp y sp las he guardado dentro del archivo `res/values/dimens.xml` (como padding_interno_tarjeta, tamano_boton_flotante o texto_titulo_barra) para enlazarlas con @dimen/ y no dejar ningún número escrito a mano en los layouts. Por último, he personalizado tanto el archivo `themes.xml` principal como el de modo noche (values-night/themes.xml) enlazando mis colores propios en los atributos

Finalmente, el aspecto de la aplicación tras ejecutarse en el emulador, es el siguiente:


13:43
Querida Aiko
Para mi yo de 2027
Desbloqueo: 15/05/2027
Promesa de año nuevo
Desbloqueo: 01/01/2028
Un desahogo de hoy
Desbloqueada: 12/03/2026
Querida Aiko, hoy me sentí cansada, pero al escribirte me sentí más ligera. Te mando un...
13:43
Nueva promesa
Escribe lo que sientes, una promesa, un desahogo...
Elegir fecha de apertura (dd/mm/aaaa)
Guardar y bloquear





6. Conclusión personal

6.1. Aprendizaje adquirido

Hacer esta práctica me ha servido sobre todo para entender cómo hay que ordenar un proyecto en Android Studio desde cero. Antes pensaba que con hacer que la pantalla quedara chula ya valía, pero ahora veo que separar los textos en el `strings.xml`, usar estilos y organizar los colores bien hace que todo sea mucho más limpio y fácil de mantener. También me ha gustado ver lo cómodo que es que Android gestione solo cosas como la internacionalización si configuras bien las carpetas de idiomas

6.2. Dificultades encontradas

Me he encontrado con varios problemas molestos. Por un lado, el ordenador tardó mucho en descargar los emuladores, además cuando arrancaba las máquinas virtuales hacía que todo fuera lento para probar los cambios. Luego, pasar los diseños exactos que tenía en Figma a Android Studio me costó mucho, porque las restricciones en los layouts a veces hacían lo que querían y tenías que estar probando mil veces con los márgenes y las medidas hasta que por fin encajaba todo como lo había pensado

7. Bibliografía

Android Developers. (s. f.). Aplicar estilos y temas. Google. https://developer.android.com/guide/topics/ui/look-and-feel/themes?hl=es

Android Developers. (s. f.). Ciclo de vida de la actividad. Google. https://developer.android.com/guide/components/activities/activity-lifecycle?hl=es

Android Developers. (s. f.). Crear iconos de aplicaciones con Image Asset Studio. Google. https://developer.android.com/studio/write/image-asset-studio?hl=es

Android Developers. (s. f.). Documentación oficial de Android Studio y desarrollo de aplicaciones móviles. Google. https://developer.android.com/?hl=es

Android Developers. (s. f.). Información general sobre elementos de diseño (Drawables). Google. https://developer.android.com/guide/topics/graphics/drawables?hl=es

Android Developers. (s. f.). Información general sobre recursos de aplicaciones. Google. https://developer.android.com/guide/topics/resources/providing-resources?hl=es

Android Developers. (s. f.). Localización de la aplicación y mejores prácticas (values-en). Google. https://developer.android.com/guide/topics/resources/localization?hl=es

Android Developers. (s. f.). Recursos de cadenas y parámetros de formato (%1$s). Google. https://developer.android.com/guide/topics/resources/string-resource?hl=es

Google Fonts. (s. f.). Catálogo de tipografías abiertas (Londrina Solid e Inder). Google. https://fonts.google.com/

Gradle Inc. (s. f.). Gradle Build Tool: Documentación oficial para compilación en Android. https://docs.gradle.org/

Instituto MEDAC - DAVANTE. (2026). Módulo de Programación Multimedia y Dispositivos Móviles (PMDM): Material didáctico oficial y guías de prácticas (Temas 1 a 3). Instituto MEDAC - DAVANTE.

Oracle. (s. f.). Documentación de Java SE y guías de programación. Oracle Corporation. https://www.oracle.com/es/java/technologies/docs.html